AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Implementacja drzewa BST w C++ z zastosowaniem GitHub

Autor:
Jakub Pacułt, Jakub Podkówka

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **4**

1.1. Struktura i implementacja drzewa BST 4

1.2. Kod źródłowy węzła 5

1.3. Zarządzanie pamięcią, pliki i testowanie 5

2. Analiza problemu **6**

2.1. Zastosowanie struktur danych i wzorców projektowych 6

2.2. Sposób działania programu i algorytmów 6

2.3. Przykład działania algorytmu wstawiania 8

2.4. Opis narzędzi wspierających projekt i kontrolę wersji 8

3. Projektowanie **9**

3.1. Środowisko i narzędzia programistyczne 9

3.2. Ustawienia kompilatora i konfiguracja projektu 9

3.3. Architektura systemu i diagram klas 10

3.4. Organizacja pracy w dwuosobowym zespole z użyciem GitHuba 10

3.5. Dokumentacja kodu źródłowego (Doxygen) 13

4. Implementacja **14**

4.1. Omówienie wybranych fragmentów kodu źródłowego 14

4.1.1. Rekurencyjne wstawianie elementu do drzewa BST 14

4.1.2. Serializacja binarna drzewa (FileManager) 15

4.2. Zestawienie kluczowych metod i operacji drzewa BST 15

4.3. Opis uzyskanych wyników i działania programu 16

5. Wnioski **17**

Literatura **18**

Spis rysunków **18**

Spis tabel **19**

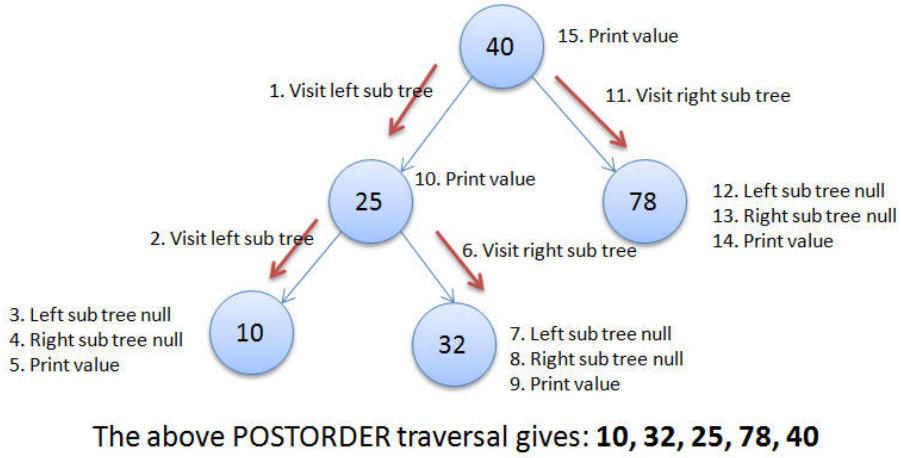
Spis listingów **20**

1. Ogólne określenie wymagań

Celem niniejszego projektu jest zaimplementowanie i przetestowanie struktury danych typu drzewo poszukiwań binarnych (BST, ang. *Binary Search Tree*) w języku C++. Oczekiwanym wynikiem jest stworzenie efektywnego kontenera dynamicznego, który umożliwia dodawanie, usuwanie, wyszukiwanie ścieżki do elementu, graficzne wyświetlanie (z uwzględnieniem metod *preorder*, *inorder* oraz *postorder*) oraz zapis do plików tekstowych i binarnych. Program ma na celu zaprezentowanie poprawnego zarządzania pamięcią dynamiczną, pracy z plikami oraz zespołowej pracy nad projektem z wykorzystaniem systemu Git i platformy GitHub.

1.1. Struktura i implementacja drzewa BST

Każdy element drzewa BST jest reprezentowany przez osobny węzeł. Węzeł przechowuje wartość całkowitą oraz dwa wskaźniki: na lewe oraz prawe poddrzewo. Właściwość drzewa BST gwarantuje, że wartości w lewym poddrzewie są mniejsze od węzła rodzica, a w prawym poddrzewie większe.



Rys. 1.1. Przykładowe drzewo oraz metoda przechodzenia po drzewie - POSTORDER

Rysunek 1.1 (s. 4) przedstawia drzewo binarne oraz metodę przechodzenia po nim - POSTORDER.

Tabela 1.1 (s. 5) przedstawia zestawienie złożoności czasowej i pamięciowej dla podstawowych operacji na drzewie BST.

Tab. 1.1. Złożoność obliczeniowa operacji w drzewie BST

Operacja	**Złożoność (średnia)**	**Złożoność (najgorsza)**	**Złożoność pamięciowa**
Dodanie elementu	$O(\log n)$	$O(n)$	$O(1)$
Wyszukanie elementu / ścieżki	$O(\log n)$	$O(n)$	$O(1)$
Usunięcie elementu	$O(\log n)$	$O(n)$	$O(1)$

1.2. Kod źródłowy węzła

Kod 1 (s. 5) przedstawia implementację struktury pojedynczego węzła, która stanowi fundamentalny budulec całego drzewa.

```
#ifndef NODE_H
#define NODE_H

struct Node {
    int data;       // Warto Ż  przechowywana w w Ź le
    Node* left;     // Wska nik na lewe poddrzewo
    Node* right;    // Wska nik na prawe poddrzewo

    // Konstruktor inicjalizuj cy w Źze Ć
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

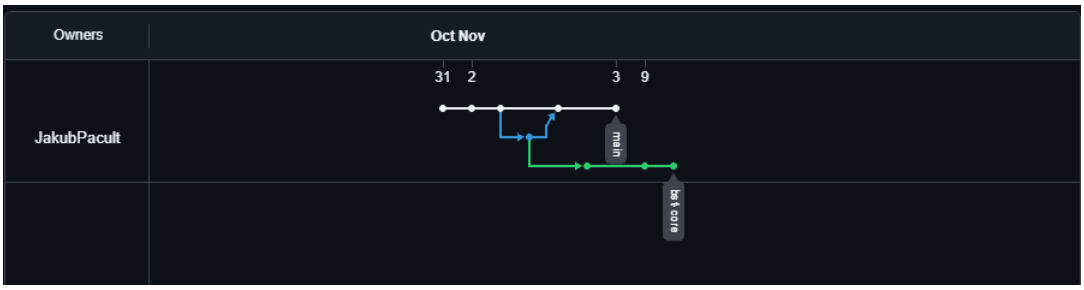
#endif
```

Listing 1. Definicja struktury węzła Node

1.3. Zarządzanie pamięcią, pliki i testowanie

Ponieważ drzewo BST alokuje pamięć dynamicznie przy każdym wstawianiu elementu, kluczowym elementem programu jest rekurencyjny destruktor oraz metoda czyszcząca (‘clear‘), które zapobiegają wyciekom pamięci. Dodatkowo projekt realizuje operacje wejścia-wyjścia poprzez klasę ‘FileManager‘, obsługującą zapis i odczyt struktury w formacie binarnym oraz budowę drzewa z plików tekstowych.

Testowanie poprawności implementacji odbywa się poprzez interaktywne menu konsolowe w pliku ‘main.cpp‘. Na rysunku 3.5 (s. 13) zaprezentowano postęp commitów gałęzi main oraz bst-core.



Wykres dla gałęzi *bst-core* oraz *main*

2. Analiza problemu

2.1. Zastosowanie struktur danych i wzorców projektowych

Drzewo poszukiwań binarnych (BST, ang. *Binary Search Tree*) jest jedną z najważniejszych hierarchicznych struktur danych w informatyce. Dzięki zachowaniu porządku kluczy (gdzie elementy mniejsze znajdują się w lewym poddrzewie, a większe w prawym), drastycznie skraca czas wyszukiwania danych w porównaniu do struktur liniowych. Znajduje szerokie zastosowanie m.in. w:

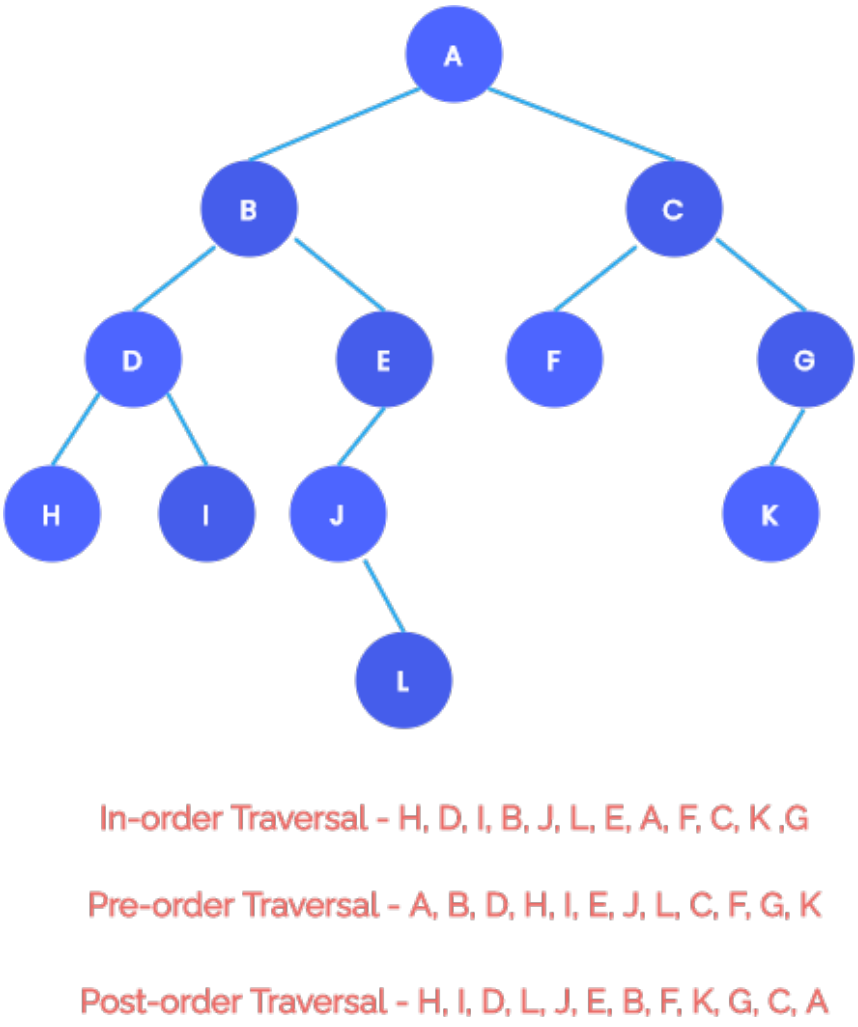
- Implementacji tablic asocjacyjnych, słowników oraz indeksów bazy danych.
- Algorytmach wyszukiwania i sortowania danych w czasie logarytmicznym.
- Reprezentacji wyrażeń algebraicznych oraz kompilacji drzew składniowych (*Abstract Syntax Tree*).
- Systemach plików i indeksowaniu katalogów dyskowych.

W ramach projektu zastosowano modułowy podział na klasy odpowiedzialne za logikę drzewa (‘BST‘), pojedyncze węzły (‘Node‘) oraz zarządzanie plikami tekstowymi i binarnymi (‘FileManager‘), co odzwierciedla zasadę separacji odpowiedzialności (*Separation of Concerns*).

2.2. Sposób działania programu i algorytmów

Program zarządza dynamiczną alokacją pamięci za pomocą wskaźników łączących węzły drzewa. Główne algorytmy zaimplementowane w projekcie obejmują:

1. **Dodawanie elementów (`insert`):** Rekurencyjne wyszukiwanie odpowiedniego miejsca w strukturze zgodnie z regułą porządku BST i alokacja nowego węzła.



Rys. 2.1. Metody przechodzenia po drzewie

2. **Usuwanie elementów (`remove`):** Obsługa trzech przypadków strukturalnych: usuwanie węzła bez dzieci, z jednym dzieckiem lub z dwojgiem dzieci (z wykorzystaniem następnika).

3. **Szukanie ścieżki (`findPath`):** Algorytm przechodzący od korzenia do zadanego węzła i zwracający pełną ścieżkę odwiedzonych wierzchołków.

4. **Przechodzenie drzewa (Traversals):** Wyświetlanie zawartości trzema metodami uporządkowanymi: *Preorder*, *Inorder* oraz *Postorder*.

5. **Operacje plikowe (`FileManager`):** Zapis tekstowy, serializacja binarna struktury oraz dynamiczna budowa drzewa na podstawie danych z pliku.

2.3. Przykład działania algorytmu wstawiania

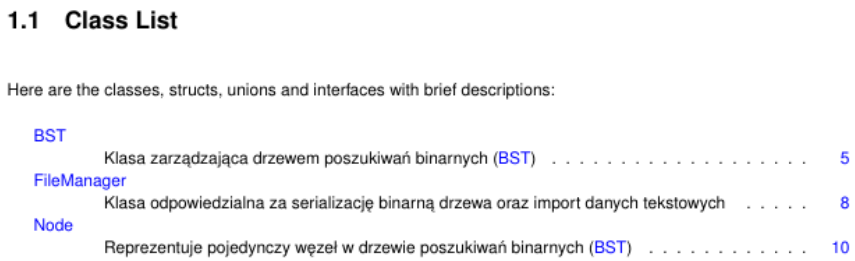
Aby zilustrować działanie algorytmu wstawiania, przeanalizujmy proces dodawania elementu o wartości 5 do pustego początkowo drzewa, a następnie wstawienia wartości 3 oraz 8.

1. **Stan początkowy:** Drzewo jest puste. Wskaźnik `root` ma wartość `nullptr`.
2. **Krok 1 (Wstawienie korzenia - 5):** Ponieważ `root == nullptr`, tworzony jest nowy węzeł [5], który staje się korzeniem drzewa.
3. **Krok 2 (Wstawienie wartości mniejszej - 3):** Wartość $3 < 5$, więc algorytm przechodzi do lewego poddrzewa korzenia. Ponieważ lewe dziecko jest puste, tworzony jest węzeł [3] jako lewy potomek węzła 5.
4. **Krok 3 (Wstawienie wartości większej - 8):** Wartość $8 > 5$, więc algorytm przechodzi do prawego poddrzewa korzenia i tworzy węzeł [8] jako prawy potomek węzła 5.
5. **Stan końcowy:** Otrzymujemy spójne drzewo BST o korzeniu w węźle 5, lewym dziecku 3 i prawym dziecku 8.

2.4. Opis narzędzi wspierających projekt i kontrolę wersji

W procesie tworzenia, testowania i zarządzania kodem projektu grupowego wykorzystano nowoczesne narzędzia inżynieryjne:

- **Git oraz GitHub:** System kontroli wersji, który posłużył do realizacji pracy w dwuosobowej grupie z wykorzystaniem gałęzi sekwencyjnych i równoległych oraz celowego wywołania i rozwiązania 6 konfliktów scalania.
- **Microsoft Visual Studio oraz MSBuild:** Środowisko i kompilator MSVC (narzędzie `cl.exe`), służące do szybkiej kompilacji i uruchamiania modułów programu.
- **Doxygen:** Narzędzie do automatycznego generowania dokumentacji technicznej na podstawie ustandaryzowanych komentarzy źródłowych.



1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BST
Klasa zarządzająca drzewem poszukiwań binarnych (BST) 5
FileManager
Klasa odpowiedzialna za serializację binarną drzewa oraz import danych tekstowych 8
Node
Reprezentuje pojedynczy węzeł w drzewie poszukiwań binarnych (BST) 10

Rys. 2.2. Przedstawienie klas w dokumentacji Doxygen

3. Projektowanie

3.1. Środowisko i narzędzia programistyczne

Proces projektowania i implementacji aplikacji drzewa BST został oparty o ustandaryzowany stos technologiczny:

- **Język programowania:**[**stroustrup**] C++, ze względu na wysoką wydajność oraz pełną kontrolę nad zarządzaniem pamięcią dynamiczną.
- **Środowisko programistyczne (IDE)**[**gamma˙design˙patterns**] **i Kompilator:** Microsoft Visual Studio Community oraz natywny kompilator MSVC (`cl.exe`) wraz z narzędziem budującym `MSBuild`.
- **System kontroli wersji:**[**github˙docs**] Git oraz platforma GitHub, wykorzystane do koordynacji pracy dwuosobowego zespołu, śledzenia historii zmian oraz obsługi gałęzi.
- **Generator dokumentacji:**[**doxygen˙site**] Doxygen, służący do automatycznego tworzenia dokumentacji technicznej na podstawie komentarzy w kodzie.
- **Dodatkowe biblioteki:**[**cppreference**] Standardowa biblioteka języka C++ (*STL*), w tym wektory oraz strumienie plików.

3.2. Ustawienia kompilatora i konfiguracja projektu

Kompilacja kodu źródłowego wymagała odpowiednich flag konfiguracyjnych w celu zapewnienia stabilności działania:

- **Flaga obsługi wyjątków (`/EHsc`):** Kluczowy parametr kompilatora MSVC włączający standardową obsługę wyjątków synchronicznych, co gwarantuje

poprawność alokacji pamięci.

- **Architektura docelowa:** Projekt skompilowany w architekturze 64-bitowej (`x64`) dla optymalnego wykorzystania zasobów systemu.
- **Konfiguracje budowania:** Wykorzystano warianty *Debug* (do testów i wykrywania wycieków pamięci) oraz *Release* (wersja ostateczna).

3.3. Architektura systemu i diagram klas

Projekt zrealizowano w oparciu o podział na wyspecjalizowane komponenty:

1. **Klasa `Node`:** Reprezentuje pojedynczy węzeł drzewa (przechowuje wartość oraz wskaźniki na lewe i prawe poddrzewo).
2. **Klasa `BST`:** Główna struktura zarządzająca logiką drzewa (dodawanie, usuwanie, przeszukiwanie *Preorder/Inorder/Postorder*, szukanie ścieżki).
3. **Klasa `FileManager`:** Odpowiada za operacje wejścia-wyjścia, w tym serializację binarną oraz odczyt danych tekstowych budujących drzewo.
4. **Moduł `main`:** Interfejs tekstowy oparty na pętli sterującej i menu użytkownika.

3.4. Organizacja pracy w dwuosobowym zespole z użyciem GitHuba

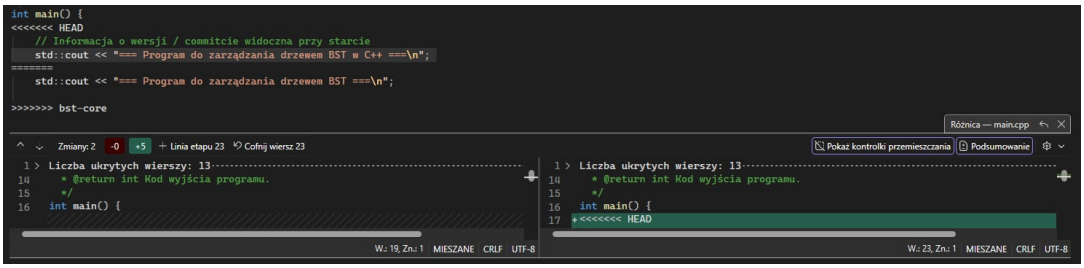
Zgodnie z wymogami projektu, proces programowania oparto na ścisłej współpracy w systemie Git, co obejmowało tworzenie gałęzi sekwencyjnych, równoległych oraz celowe wywoływanie i rozwiązywanie konfliktów.

Rozwiązywanie konfliktów (*Merge Conflicts*): Podczas scalania gałęzi celowo doprowadzono do łącznie 6 konfliktów (po 3 na każdą osobę), modyfikując dokładnie te same linie kodu w pliku `main.cpp` oraz plikach nagłówkowych. Konflikty te zostały ręcznie rozwiązane w edytorze kodu, co udokumentowano w historii repozytorium.

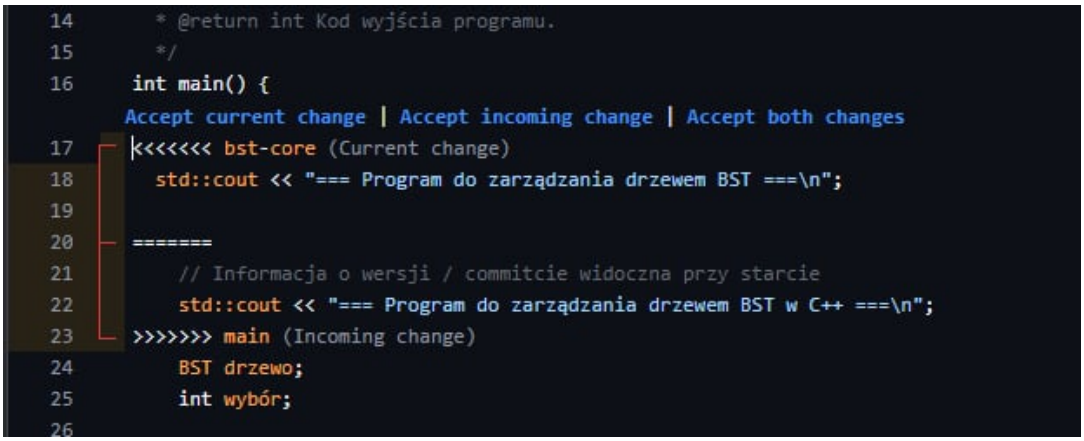
Aby zobrazować ten proces, na poniższych rysunkach przedstawiono kolejne etapy obsługi konfliktu. Początkowo system kontroli wersji informuje o niemożności automatycznego scalenia kodu (rys. 3.1). Następnie w plikach źródłowych pojawiają się specjalne znaczniki oddzielające wersje kodu z różnych gałęzi (rys. 3.2). Ostatecznie, wykorzystując wbudowane narzędzia środowiska programistycznego, programista podejmuje decyzję o wyborze odpowiedniego wariantu (rys. 3.3).

```
Auto-merging src/main.cpp
CONFLICT (content): Merge conflict in src/main.cpp
Automatic merge failed; fix conflicts and then commit the result.
```

Rys. 3.1. Komunikat o przerwaniu automatycznego scalania i wystąpieniu konfliktu w terminalu



Rys. 3.2. Podgląd znaczników konfliktu w pliku źródłowym `main.cpp`



Rys. 3.3. Rozwiązywanie konfliktu scalania pomiędzy gałęziami z wykorzystaniem narzędzi IDE

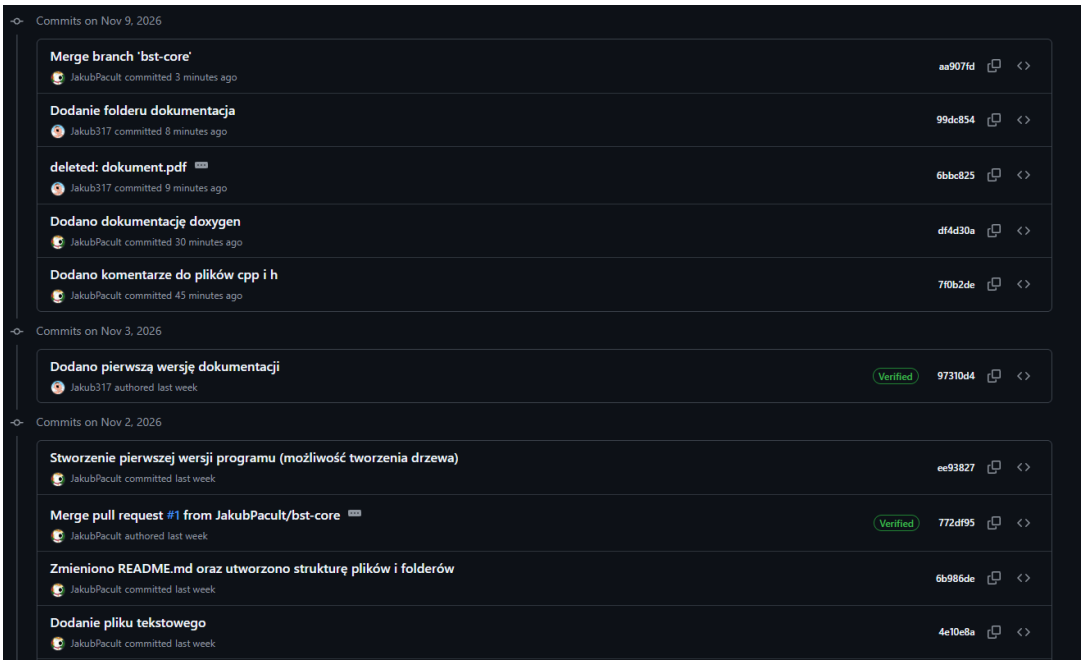
1. **Praca sekwencyjna (Gałęzie robocze):** Początkowo Osoba 1 utworzyła gałąź dedykowaną logice BST (`feature/bst-core`), zatwierdzając w niej kolejne etapy budowy klas (min. 5 commitów). Po scaleniu gałęzi do `main`, Osoba 2 utworzyła gałąź `feature/file-manager` realizującą obsługę plików binarnych i tekstowych (również min. 5 commitów) przed jej ponownym scaleniem.

2. **Praca równoległa:** W kolejnym etapie obie osoby utworzyły w tym samym punkcie czasu niezależne gałęzie robocze (`dev-osoba1` oraz `dev-osoba2`), prowadząc równoległe prace nad modyfikacją interfejsu oraz metod pomocniczych.

3. **Rozwiązywanie konfliktów (*Merge Conflicts*):** Podczas scalania gałęzi celowo doprowadzono do łącznie 6 konfliktów (po 3 na każdą osobę), modyfiku-

jąc dokładnie te same linie kodu w pliku `main.cpp` oraz plikach nagłówkowych. Konflikty te zostały ręcznie rozwiązane w edytorze kodu, co udokumentowano w historii repozytorium.

Kolejnym istotnym elementem weryfikacji pracy zespołu była kontrola liczby commitów oraz procesu ich zatwierdzania. Na rysunku 3.5 (s. 13) przedstawiono historię commitów obu członków grupy.



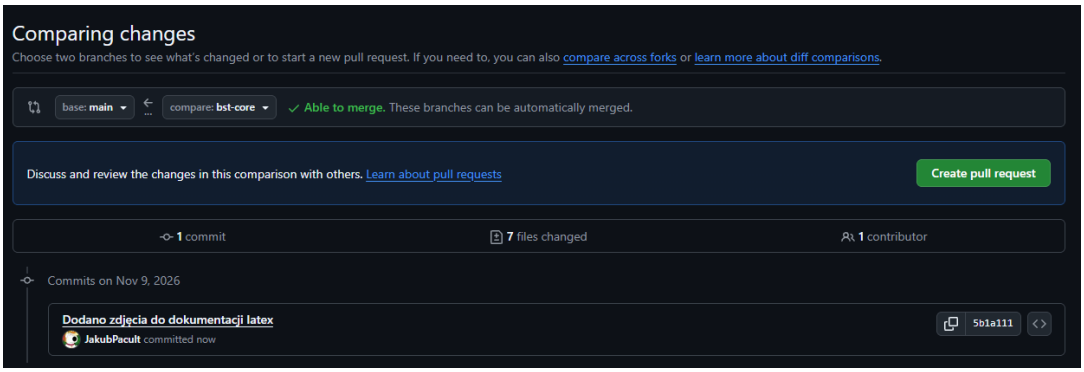
Rys. 3.4. Historia commitów poszczególnych autorów w projekcie, dodatkowo widoczny merge gałęzi oraz pull-request

W celu spełnienia wymagań dotyczących pracy zespołowej, kluczowe było również przećwiczenie procedury łączenia kodu i obsługi sytuacji spornych. Rysunek 3.6 (s. 13) przedstawia sposób tworzenia pull request'a. Umożliwia on zapoznanie się innym członkom projektu ze zmianami, które chce wprowadzić programista.

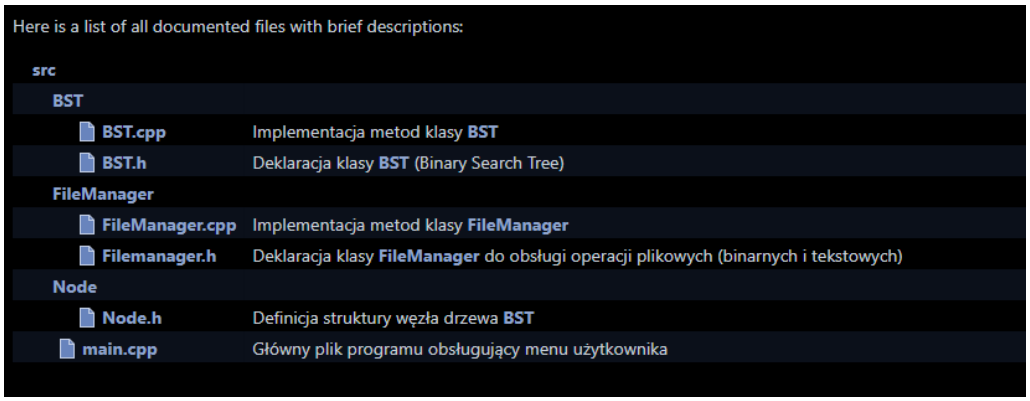
3.5. Dokumentacja kodu źródłowego (Doxygen)

Do wygenerowania dokumentacji technicznej wykorzystano program Doxygen. Kod źródłowy został opatrzony ustandaryzowanymi komentarzami zgodnymi ze składnią narzędzia (używając znaczników takich jak `@file`, `@brief`, `@param`, `@return`).

Na rysunku 4.1 (s. 15) przedstawiono wygenerowaną stronę dokumentacji HTML, która przedstawia strukturę plików projektu.



Rys. 3.5. Tworznie PR (*Pull request*) na GitHubie



Rys. 3.6. Wykaz plików projektu w doxygen

4. Implementacja

4.1. Omówienie wybranych fragmentów kodu źródłowego

Implementacja projektu została podzielona na modularne pliki nagłówkowe oraz pliki źródłowe, co zwiększa czytelność kodu oraz ułatwia jego konserwację. Poniżej przedstawiono kluczowe fragmenty oprogramowania realizujące najważniejsze założenia algorytmiczne i architektoniczne.

4.1.1. Rekurencyjne wstawianie elementu do drzewa BST

Kluczową operacją w strukturze BST jest wstawianie nowego elementu zgodnie z zasadą porządku (wartości mniejsze w lewym poddrzewie, większe w prawym). Poniższy fragment kodu z pliku `BST.cpp` przedstawia rekurencyjne podejście do tego zadania:

```
1 Node* BST::insertRecursive(Node* current, int value) {
2     if (current == nullptr) {
```

```
3         return new Node(value); // Utworzenie nowego w Źz Ća na
      w Ća Żciwym miejscu
4     }
5     if (value < current->data) {
6         current->left = insertRecursive(current->left, value);
7     } else if (value > current->data) {
8         current->right = insertRecursive(current->right, value);
9     }
10    return current;
11 }
```

Listing 2. Rekurencyjna metoda insertRecursive

```
=== MENU DRZEWA BST ===
1. Dodaj element
2. Usun element
3. Usun cale drzewo
4. Szukaj drogi do elementu
5. Wyswietl drzewo
6. Zapisz drzewo do pliku tekstowego
7. Wczytaj liczby z pliku tekstowego do drzewa
8. Zapisz drzewo do pliku binarnego
9. Odczytaj drzewo z pliku binarnego
0. Wyjście
Wybierz opcje: 1
Podaj liczbie do dodania: 3
Dodano.
```

Rys. 4.1. Menu progremu

4.1.2. Serializacja binarna drzewa (FileManager)

Zgodnie z wymaganiami projektu, obsługa plików binarnych została wydzielona do osobnej klasy `FileManager`. Poniższy kod prezentuje metodę rekurencyjnego zapisu binarnego struktury z wykorzystaniem podejścia *Preorder*:

```
1 void FileManager::serializeBinary(Node* current, std::ofstream&
      outFile) {
2     bool hasNode = (current != nullptr);
3     outFile.write(reinterpret_cast<char*>(&hasNode), sizeof(hasNode
      ));
4     if (hasNode) {
5         outFile.write(reinterpret_cast<char*>(&(current->data)),
      sizeof(current->data));
6         serializeBinary(current->left, outFile);
```

```
7           serializeBinary(current->right, outFile);
8       }
9   }
```

Listing 3. Zapis binarny drzewa w klasie FileManager

4.2. Zestawienie kluczowych metod i operacji drzewa BST

W celu usystematyzowania funkcjonalności programu, w tabeli 4.1 zestawiono najważniejsze metody zaimplementowane w klasie `BST` oraz `FileManager` wraz z ich krótkim opisem i zwracanym typem.

Tab. 4.1. Zestawienie głównych metod zaimplementowanych w projekcie

Klasa	Nazwa metody	Opis działania
`BST`	`insert(int val)`	Dodaje nową wartość do drzewa zgodnie z porządkiem BST.
`BST`	`remove(int val)`	Usuwa węzeł o zadanym kluczu (obsługuje 3 przypadki dzieci).
`BST`	`findPath(int val, vector& path)`	Znajduje ścieżkę od korzenia do zadanego elementu.
`BST`	`display(int method)`	Wyświetla drzewo metodą Preorder, Inorder lub Postorder.
`FileManager`	`saveBinary(...)`	Zapisuje strukturę drzewa do pliku w formacie binarnym.
`FileManager`	`loadTextToBST(...)`	Wczytuje sekwencję liczb z pliku tekstowego do drzewa.

4.3. Opis uzyskanych wyników i działania programu

W wyniku poprawnej kompilacji i uruchomienia programu (zarówno za pomocą skryptu konsolowego MSVC `cl.exe`, jak i w środowisku Visual Studio) otrzymano w pełni stabilną aplikację konsolową. Uzyskane rezultaty obejmują:

1. **Dynamiczne zarządzanie pamięcią:** Program poprawnie alokuje pamięć na stercie dla węzłów drzewa oraz – dzięki rekurencyjnemu destruktorowi i metodzie `clear()` – całkowicie ją zwalnia przy zamknięciu programu, eliminując wycieki pamięci (*memory leaks*).

2. **Poprawność operacji plikowych:** Moduł `FileManager` bezbłędnie realizuje zapis i odczyt struktur binarnych oraz umożliwia elastyczne budowanie drzewa na podstawie surowych danych z plików tekstowych (zarówno do struktur pustych, jak i już istniejących).

3. **Interaktywny interfejs menu:** Pętla sterująca w pliku `main.cpp` zapewnia czytelną obsługę wszystkich wymagań projektowych oraz poprawnie zabezpiecza przed błędami użytkownika.

4. **Kompletna dokumentacja:** Użycie ustandaryzowanych komentarzy Doxygen w połączeniu z odpowiednią konfiguracją ścieżek (*STRIP_FROM_PATH*) pozwoliło na wygenerowanie czytelnej dokumentacji technicznej projektu.

5. Wnioski

Przeprowadzony projekt pozwolił na praktyczne zastosowanie zaawansowanych koncepcji programowania obiektowego, struktur dynamicznych oraz systemów kontroli wersji w języku C++. Realizacja zadania w dwuosobowym zespole dostarczyła kilku istotnych wniosków o charakterze technicznym i merytorycznym:

1. **Zalety hierarchicznych struktur danych:** Drzewo poszukiwań binarnych (BST) udowodniło swoją efektywność w scenariuszach wymagających szybkiego wyszukiwania, wstawiania i usuwania danych. Dzięki strukturalnemu uporządkowaniu kluczy, operacje te wykonują się średnio w czasie logarytmicznym $O(\log n)$, co stanowi znaczącą optymalizację w porównaniu do struktur liniowych.

2. **Modułowość i separacja odpowiedzialności:** Podział projektu na dedykowane klasy (`Node`, `BST`, `FileManager`) oraz osobną obsługę interfejsu w pliku `main.cpp` znacząco podniósł czytelność kodu, ułatwił testowanie poszczególnych komponentów oraz pozwolił na bezkonfliktowy podział zadań w zespole.

3. **Praktyczna nauka pracy z Gitem i rozwiązywania konfliktów:** Realizacja wymogów związanych z gałęziami sekwencyjnymi, równoległymi oraz celowym wywołaniem i rozwiązaniem 6 konfliktów scalania pozwoliła na głębokie zrozumienie mechanizmów działania systemu Git. Praca ta odzwierciedliła realne środowisko projektowe, w którym koordynacja zmian kodu przez wielu programistów wymaga precyzji i znajomości narzędzi scalających.

4. **Zarządzanie pamięcią i operacje plikowe:** Ręczna alokacja pamięci w języku C++ wymaga dyscypliny – implementacja rekurencyjnego destruktora oraz metody `clear()` skutecznie zapobiegła wyciekom pamięci (*memory leaks*). Dodatkowo, wdrożenie klasy `FileManager` umożliwiło niezawodną serializację binarną oraz elastyczny import danych z plików tekstowych.

5. **Automatyzacja dokumentacji:** Wykorzystanie ustandaryzowanych komentarzy Doxygen w połączeniu z konfiguracją ścieżek względnych (*STRIP_FROM_PATH*) pozwoliło na szybkie i czytelne wygenerowanie pełnej dokumentacji technicznej projektu w formatach HTML oraz LaTeX.

Podsumowując, cel projektu został w pełni osiągnięty. Powstała aplikacja jest stabilna, poprawna algorytmicznie, spełnia wszystkie kryteria stawiane przez prowadzących oraz stanowi solidną podstawę do dalszego rozwoju i zgłębiania tajników inżynierii oprogramowania.

Spis rysunków

1.1. Przykładowe drzewo oraz metoda przechodzenia po drzewie - POSTORDER 4
2.1. Metody przechodzenia po drzewie 7
2.2. Przedstawienie klas w dokumentacji Doxygen 9
3.1. Komunikat o przerwaniu automatycznego scalania i wystąpieniu konfliktu w terminalu 11
3.2. Podgląd znaczników konfliktu w pliku źródłowym `main.cpp` 11
3.3. Rozwiązywanie konfliktu scalania pomiędzy gałęziami z wykorzystaniem narzędzi IDE 11
3.4. Wykres gałęzi oraz commitów w repozytorium GitHub 12
3.5. Historia commitów poszczególnych autorów w projekcie, dodatkowo widoczny merge gałęzi oraz pull-request 13
3.6. Tworznie PR (*Pull request*) na GitHubie 13
3.7. Wykaz plików projektu w doxygen 14
4.1. Menu progremu 15

Spis tabel

1.1. Złożoność obliczeniowa operacji w drzewie BST 5

4.1. Zestawienie głównych metod zaimplementowanych w projekcie 16

Spis listingów

1. Definicja struktury węzła Node 5
2. Rekurencyjna metoda insertRecursive 14
3. Zapis binarny drzewa w klasie FileManager 15